hexens x MINT

Security Review Report for Mint.io

August 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Newly staked funds can be unstaked immediately
    - A pending withdrawal request has no cost, allowing accounts to pre-arm the cooldown and exit instantly
    - Delayed withdrawal request behavior is inconsistently documented
    - userRewardPerWeightPaid update can be moved into the if block to save gas
    - Matured positions continue earning rewards after unlock

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit covers MNTD, a reward and utility token used within the Mint.io casino ecosystem on Robinhood. MNTD can be distributed as casino rewards, used by applications for progression and utility, or held and traded as an ERC-20 token.

The review was conducted over a 1.5-week period and included a comprehensive analysis of the relevant smart contracts, covering the fixed token issuance and launch distribution flows, as well as two independent staking products designed for different user groups.

During the assessment, we did not identify any critical or high-severity issues. We identified one medium-severity finding that could allow newly staked tokens to bypass the configured unstaking cooldown and be withdrawn earlier than intended. In addition, we identified one low-severity finding and three informational issues.

All identified issues were either remediated or acknowledged by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

▪ Review Led by

Trung Dinh, Lead Security Researcher

▪ Scope

The analyzed resources are located on:

https://github.com/mintdotio/mint-contracts/tree/38de223937dcf2776b859cbad1a965d9d6ee1128

The analyzed resources are located on:

https://github.com/mintdotio/mint-contracts/tree/a81fffe8afc147aefa64adaed434abfaa3d15771

▪ Changelog

17 August 2026	Audit start
27 August 2026	Initial report
2 September 2026	Revision received
3 September 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

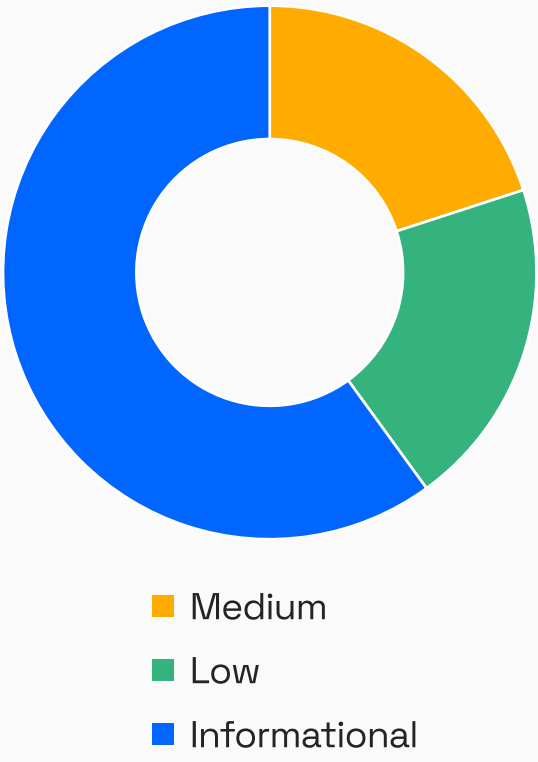
- **Issue Symbolic Codes**

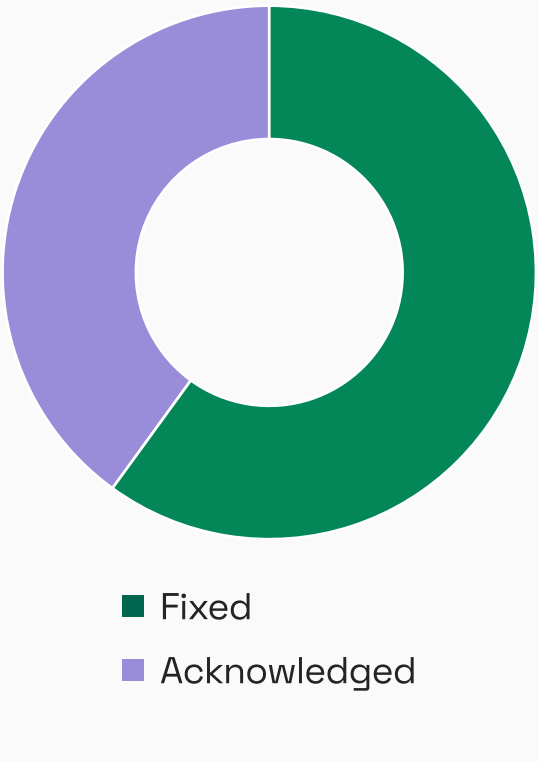
Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	1
Low	1
Informational	3
Total:	**5**





6. Weaknesses

This section contains the list of discovered weaknesses.

MINT1-1 | Newly staked funds can be unstaked immediately

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

src/flexible-staking/FlexibleStaking.sol#L594

Description:

This issue assumes that **unstakingCooldown** is greater than zero.

FlexibleStaking.unstakeAllUnlocked() is used to withdraw all currently unlocked stake after the configured **unstakingCooldown**. When a cooldown is enabled, the function creates an unstake request with **amount** set to **0**:

```
function unstakeAllUnlocked() external nonReentrant {
    ...

    if (unstakingCooldown > 0) {
        ...
        _createUnstakeRequest(msg.sender, /* amount */ 0);
        return;
    }

    ...
}
```

The request is therefore stored as:

```
unstakeRequestOf[account] = UnstakeRequest({
    amount: 0,
    executableAt: executableAt
});
```

The issue is that storing **amount = 0** causes the request to represent the user's entire balance at execution time, rather than the balance that existed when the request was created.

As a result, a user can make a new stake after creating the unstake request and then withdraw that newly staked amount as soon as the original cooldown expires, without waiting for the full **unstakingCooldown**.

For example:

1. At time **T**, Alice calls **unstakeAllUnlocked()** to unstake her existing **X** tokens:
   - **unstakeRequestOf[Alice].amount = 0**
   - **unstakeRequestOf[Alice].executableAt = T + unstakingCooldown**
2. At **T + unstakingCooldown - 1**, Alice stakes an additional **Y** tokens.
3. At **T + unstakingCooldown**, a keeper calls **executeUnstake()** for Alice. Because the request stores **amount = 0**, the function withdraws all of Alice's current balance (**X + Y**) instead of only the **X** tokens that were eligible when the request was created.

Therefore, the newly staked **Y** tokens can bypass the configured **unstakingCooldown** and be withdrawn earlier than intended.

Remediation:

Consider modifying the function **unstakeAllUnlocked()** as follows:

```
function unstakeAllUnlocked() external nonReentrant {
    ...

    if (unstakingCooldown > 0) {
        ...

        uint256 unlockedBalance = _unlockedBalance(msg.sender, SMNTD_TOKEN.balanceOf(msg.sender));
        if (unlockedBalance == 0) revert InsufficientUnlockedStake(0);
--      _createUnstakeRequest(msg.sender, 0);
++.     _createUnstakeRequest(msg.sender, unlockedBalance);
        return;
    }
    ...
}
```

MINT1-4 | A pending withdrawal request has no cost, allowing accounts to pre-arm the cooldown and exit instantly

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

src/flexible-staking/FlexibleStaking.sol#L672-L679
src/flexible-staking/FlexibleStaking.sol#L611-L621

Description:

When the cooldown is enabled, creating an unstake request only records the request. It does not transfer, burn, or reserve any of the account's stake:

```
function _createUnstakeRequest(address account, uint256 amount) private {
    uint256 executableAt = block.timestamp + unstakingCooldown;
    unstakeRequestOf[account] = UnstakeRequest({
        amount: amount,
        executableAt: executableAt
    });
    emit UnstakeRequested(account, amount, executableAt);
}
```

This is intentional. Specification 10.2 states that creating a request must not reserve value and that rewards continue accruing based on the existing stake weight during the cooldown period.

However, this also means that a pending request has no cost or restriction. The stake continues earning rewards at full weight, **cancelUnstakeRequest()** is free, and staking and compounding remain available. Once the request matures, it does not expire and can be executed at any time.

As a result, an account can call **unstakeAllUnlocked()** immediately after its first deposit and simply leave the request pending. After one cooldown period, the account has a permanently matured withdrawal request that can be executed at any time without incurring any cost.

Therefore, the cooldown only affects accounts that have not created a request in advance. It does not prevent an account from pre-arming the cooldown and later exiting immediately.

Similarly, an account can stake enough to reach a higher reward tier, wait for the pre-existing request to mature, and then exit immediately after receiving the benefit.

Remediation:

Consider excluding the requested amount from the account's reward weight while the request is pending and restore it on cancellation, so that keeping a request open has a real cost.

Alternatively, give the request an execution window, so that it can only be executed between **executableAt** and **executableAt + window** and must be recreated afterwards.

MINT1-2 | Delayed withdrawal request behavior is inconsistently documented

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

docs/SPECIFICATION.md?plain=1#L339

Description:

The specification defines different behaviors for delayed withdrawal request creation.

Section 6 and section 8.3 require rewards to be settled and compounded and the account to be synchronized to the latest reward curve.

> Stake, compound, withdrawal request, and withdrawal execution MUST settle the account under its old weight before compounding or changing balance, then calculate its final weight using the latest curve and store the latest version.
>
> Immediate withdrawal, delayed request creation, and delayed execution MUST settle and compound before continuing. Request creation stores no withdrawal yet, but synchronizes the account to the latest curve.

However, section 10.2 states that request creation should not modify rewards, sMNTD, or weight. Section 15 similarly requires delayed requests to be non-mutating, with compounding deferred until execution.

> When `unstakingCooldown > 0`, request creation MUST NOT update global reward, settle the account, compound, mint or burn sMNTD, change weight, change liabilities, or reserve value.
>
> Flexible tests MUST cover:
> - delayed request non-mutation and execution-time compounding;

The implementation follows sections 6 and 8.3. When the cooldown is enabled, unstake settles and compounds the account before creating the request:

```
function unstake(
    uint256 amount
) external nonReentrant {
    _requireWithdrawalsUnfrozen(msg.sender);
```

```
    if (amount == 0) revert AmountZero();
    if (unstakingCooldown > 0) {
        _updateReward(msg.sender);
        _compoundSettled(msg.sender);
        uint256 liveBalance = SMNTD_TOKEN.balanceOf(msg.sender);
        uint256 unlockedBalance = _unlockedBalance(msg.sender, liveBalance);
        if (amount > unlockedBalance) revert InsufficientUnlockedStake(unlockedBalance);
        _createUnstakeRequest(msg.sender, amount);
        return;
    }

    _clearUnstakeRequest(msg.sender);
    _settleCompoundAndWithdraw(msg.sender, amount, false);
}
```

Integrations that follow sections 10.2 or 15 may represent rewards, sMNTD balances, curve versions, or reward weights differently from the on-chain state during the cooldown.

Remediation:

- Update sections 10.2 and 15 to match the implemented request-time settlement, compounding, and curve synchronization behavior.
- Clarify that request creation does not burn sMNTD, transfer MNTD, reduce liabilities, or reserve withdrawal value.

MINT1-3 | userRewardPerWeightPaid update can be moved into the if block to save gas

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/flexible-staking/FlexibleStaking.sol#L904

Description:

The **FlexibleStaking._updateReward()** function currently updates **userRewardPerWeightPaid[account]** regardless of whether the user's reward state has changed:

```
function _updateReward(
    address account
) private {
    _updateGlobalReward();

    uint256 current = rewardPerWeightStored;
    uint256 delta = current - userRewardPerWeightPaid[account];

    if (delta > 0) {
        storedRewardsOf[account] += Math.mulDiv(
            rewardWeightOf[account],
            delta,
            REWARD_PRECISION
        );
    }

    userRewardPerWeightPaid[account] = current;
}
```

When **delta == 0**, **current** is already equal to **userRewardPerWeightPaid[account]**, making the storage write redundant.

Remediation:

Consider moving the update inside the **if** block to avoid the unnecessary storage operation:

```
function _updateReward(
    address account
) private {
    _updateGlobalReward();

    uint256 current = rewardPerWeightStored;
    uint256 delta = current - userRewardPerWeightPaid[account];

    if (delta > 0) {
        storedRewardsOf[account] += Math.mulDiv(
            rewardWeightOf[account],
            delta,
            REWARD_PRECISION
        );
+       userRewardPerWeightPaid[account] = current;
    }

-   userRewardPerWeightPaid[account] = current;
}
```

This preserves the existing behavior while avoiding a redundant storage write when **delta == 0**.

MINT1-5 | Matured positions continue earning rewards after unlock

Acknowledged

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/locked-staking/LockedStaking.sol

Description:

A position's **lsMNTD** weight is only burned when it is withdrawn. It remains in **totalRewardWeight()** after **unlockTime**, allowing a position with no remaining lock commitment to continue earning rewards.

For example, if Alice and Bob each stake 1,000 for 100 days and the reward schedule lasts 200 days:

- Days 0–100: rewards are split 50/50.
- At day 100: both positions mature, but both still have weight.
- If Alice calls **withdrawExpired(bob)**, Bob's weight is removed.
- Alice then receives 100% of the rewards for days 100–200.

Thus, matured positions dilute still-locked positions, while the timing of **withdrawExpired()** determines when that dilution ends.

Remediation:

Ensure matured positions are removed from the reward-weight denominator at **unlockTime**, rather than relying on the keeper to call **withdrawExpired()**.

hexens x MINT